

1. Login Management System

Question

Create a complete **Login Management System** in Python.

The system should allow users to:

1. Register a new account
 2. Login into the system
 3. View all registered users
 4. Exit the program
-

Requirements

Variables and Data Types

- Store:
 - Username
 - Password
 - Age
 - Email
 - Use proper datatypes.
-

Input and Output

- Take all inputs from user.
 - Display outputs in proper format.
-

Operators

- Use comparison operators for:
 - Username checking
 - Password matching
 - Use logical operators:
 - and
 - or
-

Conditions

- If username already exists:
 - Display error message
 - If password is incorrect:
 - Show invalid login message
-

Loops

- Use a menu-driven loop:
 - Program should continue until user chooses Exit.
-

Lists and Dictionaries

- Store all users using:

users = []

- Each user should be stored as dictionary:

```
{  
  "username": "",  
  "password": "",  
  "email": ""  
}
```

Functions

Create separate functions for:

- Registration
 - Login
 - Display users
-

Lambda Function

Use lambda function to:

- Check password length
 - OR sort usernames alphabetically
-

Exception Handling

Handle:

- Empty input
 - Wrong datatype
 - Invalid menu choice
-

File Handling

Store registered users in:

users.txt

When program starts:

- Load all users from file

When new user registers:

- Save data into file

2. Student Management System

Question

Create a complete **Student Management System** using Python.

The system should allow:

1. Add Student
 2. View Students
 3. Search Student
 4. Update Student Marks
 5. Delete Student
 6. Exit
-

Requirements

Variables and Data Types

Store:

- Student ID
- Name
- Age

- Course
 - Marks
-

Operators

Use operators to:

- Calculate percentage
 - Calculate grade
-

Conditions

Use:

- if
- elif
- else

For grading system:

Marks	Grade
-------	-------

80+	A
-----	---

70+	B
-----	---

60+	C
-----	---

Below 60	Fail
----------	------

Loops

Use loops to:

- Display all students
 - Search records
 - Keep menu running
-

Data Structures

Use:

- List
- Dictionary

- Tuple
- Set

Example:

```
student = {  
    "id": 101,  
    "name": "Ali",  
    "marks": 85  
}
```

Functions

Create separate functions for:

- Add student
- Search student
- Update student
- Delete student
- Display students

Lambda Functions

Use lambda to:

- Sort students by marks
- Sort students by names

Exception Handling

Handle:

- Invalid marks
- Invalid student ID
- Duplicate IDs

File Handling

Store records in:

students.txt

Program should:

- Save records
- Load records automatically

-
1. Create a function that takes a list of numbers as argument and returns the sum of all even numbers present in the list.
 2. Write a program that takes student marks as input, stores them in a list, and uses a function to calculate the average marks and grade.
 3. Create a function that accepts a string and returns the number of uppercase letters, lowercase letters, and digits present in it.
 4. Write a program that takes 10 numbers from the user using loops and stores them in a list. Create a function to display the largest and smallest number.
 5. Create a menu-driven calculator using functions and loops that performs addition, subtraction, multiplication, and division.
 6. Write a function that accepts a list of names and returns a new list containing names with length greater than 5.
 7. Create a lambda function that checks whether a given number is divisible by both 3 and 5.
 8. Use lambda function with map() to convert a list of strings into uppercase.
 9. Write a program using lambda and filter() to display only odd numbers from a list.
 10. Create a list of tuples containing employee names and salaries, then sort them using lambda according to salary.
 11. Use lambda function with map() to calculate squares of numbers from 1 to 20.
 12. Write a program using lambda function to check whether a word starts with a vowel or consonant.
 13. Create a login system where usernames and passwords are stored in a dictionary. Use loops and conditions to validate login credentials.
 14. Write a program that stores product names and prices in a dictionary. Create functions to add products, update prices, and calculate total bill.
 15. Create a student attendance system using lists and dictionaries where attendance percentage is calculated using functions.
 16. Write a program that takes marks of 5 subjects, stores them in a list, and calculates total marks, percentage, and grade using functions.
 17. Create a banking system using functions where users can deposit, withdraw, and check balance. Prevent withdrawal if balance is insufficient.

18. Write a program that takes a sentence from the user and uses functions to count vowels, consonants, spaces, and digits.
19. Create a function that accepts a list and removes duplicate values using sets.
20. Write a program that uses exception handling to prevent division by zero and invalid integer input.
21. Create a file handling program that writes student names and marks into a file and then reads and displays all records.
22. Write a program that stores employee data in a dictionary and uses loops to display employees whose salary is greater than 50000.
23. Create a menu-driven program using loops and functions to manage a shopping cart system.
24. Write a program that uses nested loops to print patterns and stores pattern rows in a list.
25. Create a function that takes a tuple of numbers and returns the sum of prime numbers only.
26. Write a program using lambda and sorted() to sort a list of dictionaries according to student marks.
27. Create a password validation system using conditions and functions. Password must contain:
 - At least 8 characters
 - One uppercase letter
 - One digit
28. Write a program that reads a text file and counts:
 - Total words
 - Total lines
 - Total characters